Metodología de la Programación

Grado en Ingeniería Informática

Curso 2022/2023

Apellidos1, Nombre1

Apellidos2, Nombre2

Apellidos3, Nombre3

Apellidos4, Nombre4

Proyecto Modularidad

Nombre del proyecto

Índice general

Índice general **1**
1. Introducción 2
2. Documentación de usuario 4
   2.1. Descripción funcional 4
   2.2. Tecnología 4
   2.3. Manual de instalación 4
   2.4. Acceso al sistema 5
   2.5. Manual de referencia 6
   2.6. Guía del operador 7
3. Documentación de sistema 8
   3.1. Especificación del sistema 8
   3.2. Módulos 8
   3.3. Plan de prueba 8
4. Documentación del código fuente 11

1. Introducción

Este es un ejemplo de plantilla para la generación de la documentación asociada al desarrollo de un programa. Se puede usar como guía, pero puede sufrir todas las modificaciones que considere oportunas para mejorar su documento.

Es necesario incluir una portada, dispone de cuatro ejemplos. Puede comprobar el resultado de cada uno de ellos en los ficheros PDF. En el fichero .tex está por defecto el ejemplo de Portada1.

Se debe añadir un índice. Puede observar en el código cómo se puede incluir en la sección destinada a tal fin.

A lo largo del documento también puede surgir la necesidad de añadir una imagen. Puede ver cómo hacerlo en el código de las portadas.

LaTeXtambién le permite incluir código. Fíjese en el código fuente cómo está delimitado. El resultado para el típico ejemplo *Hola mundo* en C es el siguiente:

```
/* Programa: Hola mundo */

#include <stdio.h>

int main()
{
    printf( "Hola mundo." );
    system("PAUSE");
    return 0;
}
```

Puede consultar tutoriales, libros o blogs para profundizar más en el aprendizaje de LaTeX, resulta muy interesante por la gran ventaja que ofrece de proporcionar documentos con una alta calidad. Sea cual el sistema operativo tenga en cuenta que va a neccesitar:

1. Editor de texto para generar el fichero .tex con el código
2. Distribución de LaTeXpara compilar el fichero .tex y generar el fichero .pdf
3. Visor de documentos para poder visualizar el fichero .pdf generado

Las siguientes secciones indican cada una de las partes que se debe incluir en la documentación del programa. Hay que tener en cuenta que se trata de una tarea tediosa pero muy necesaria para realizar un buen mantenimiento

del programa. Para cualquier modificación del código hay que entender el mismo, siendo de gran ayuda la documentación.

Recuerde que en la documentación del programa se distinguen dos partes claramente diferenciadas:

- Documentación interna. Documentación contenida en las líneas de comentarios del programa.
- Documentación externa. Documentación que incluye el análisis, diagramas de flujos y/o pseudocódigos, manuales de usuario con instrucciones para ejecutar el programa e interpretar los resultados. Dentro de esta se distingue a su vez:
    - Documentación del usuario
    - Documentación del sistema

En las siguientes secciones se describirá el contenido a incluir en cada uno de estos tipos de documentación. El objetivo que se debe marcar en la redacción de las mismas debe ser el de conseguir un texto claro, completo y conciso, utilizando para ello construcciones gramaticales y ortográficas correctas.

2. Documentación de usuario

Documentación relacionada con las funciones del sistema pero no se hace referencia a nada relativo a su implementación. Está orientado a las personas que van a usar el sistema no a los que lo van a mantener.

2.1. Descripción funcional

- Debe describir de forma simple el propósito del sistema.
- En esta sección se debe incluir una breve descripción funcional sobre lo que hace y no hace el sistema.
- Se pueden incluir pequeños ejemplos para dar una visión general de las características del sistema.

2.2. Tecnología

Las tecnologías y herramientas usadas para el desarrollo de este proyecto de juego de escape-room son:

- Git: Usado para la administración de los archivos del proyecto, control de versión, y compartido de archivos para trabajo en equipo.
- Visual Studio Code: Editor de código con integración de Git para ver los cambios realizados y resolver conflictos donde varios miembros del equipo suban modificaciones que colisionen una con la otra.
- Librerías: Se usan las librerías del estándar ANSI C, las cuales son empleadas para constituir módulos que realizan las distintas tareas que requiere el proyecto.

2.3. Manual de instalación

Para la instalación de nuestra implementación de ESI-Escape, el usuario deberá ejecutar desde la terminal **(¡MUY IMPORTANTE, hacerlo estando dentro del directorio del proyecto, para asegurar la carga de archivos con las rutas correctas!)** "run" (para directamente ejecutar el proyecto si está ya compilado en la carpeta "bin"), "run_and_compile" (para compilar el proyecto y luego ejecutarlo) o "compile" (para solo compilarlo en la carpeta "bin"). Se usarán los archivos .bat si el usuario se encuentra en el sistema operativo Windows y los archivos .sh si se encuentra en Linux.

Para la compilación, es necesario que el usuario tenga instalado "make" mediante la forma preferida y disponible del usuario (en Linux lo instalamos usando el administrador de paquetes preferido para instalar "build_essential", pero cabe notar que la instalación de esta herramienta varía dependiendo del tipo de distribución, y muchas veces viene preinstalada), y en Windows que tenga instalado "MinGW" con su carpeta "bin" añadida al PATH para la llamada que hacen "run_and_compile" y "compile" de "mingw32-make", el cual cumple la misma función que "make": compilar todo el proyecto con sus archivos de cabecera .h y de código .c.

(Los archivos .txt de datos de entrada y de guardado de partida y jugadores se encuentran en el directorio "data").

2.4. Acceso al sistema

Al entrar al programa, se le pedirá al usuario introducir su nombre de acceso y contraseña. Si estos no son encontrados, se le ofrece el usuario registrarse, pidiendo adicionalmente un nombre completo para este. Una vez iniciado sesión, el usuario verá el menú de inicio de ESI-ESCAPE, con las opciones de comenzar una Nueva partida, Cargar partida y Salir (para salir del juego de vuelta al sistema operativo). Durante una partida, la opción "Volver"te permitirá salir al menú de inicio para entonces dejar el juego.

El usuario se puede encontrar con el problema de encontrarse mensajes de alerta con archivos los cuales no pudieron ser cargados. En esos casos, la solución radica en ejecutar "run" o "run_and_compile" situándonos con la terminal en la carpeta del proyecto (es decir, la misma que contiene dichos archivos), para que así ESI-Escape encuentre los archivos .txt.

También puede ocurrir que el sistema operativo no encuentre "bin/esi-escape" o "mingw32-make" o "make".

1. Si no se encuentra "bin/esi-escape" al ejecutar "run", significa que no está compilado, o no se está ejecutando "run" o "run_and_compile'desde la carpeta del juego'. Se soluciona ejecutando "run", "run_and_compile" o "compile" desde la carpeta del juego, y asegurándonos de que el ejecutable está presente en "bin" haciendo "run_and_compile" o "compile".

2. Si no se encuentra "mingw32-make", instalar MinGW en tu equipo Windows, o si estás usando Linux, es recomendable instalar "make" en su lugar mediante la forma preferida y disponible del usuario (en Linux lo instalamos usando el administrador de paquetes preferido para instalar "build_esential").

3. Si no se encuentra "make", y estando en Linux, lo instalamos mediante la forma preferida y disponible del usuario (en Linux lo instalamos usando el administrador de paquetes preferido para instalar "build_essential", pero cabe notar que la instalación de esta herramienta varía dependiendo del tipo de distribución, y muchas veces viene preinstalada).

2.5. Manual de referencia

La interfaz de juego de ESI-Escape permite una navegación fluida entre las salas y sus objetos y puzles; permitiendo realizar diversas acciones sobre estos.

- Las opciones de "Describir sala", "Examinar (objetos y salidas)" e "Inventario" mostrarán información sobre lo que sus nombres indican, para luego pedir al usuario una pulsación para volver al menú de juego.

- "Entrar en otra sala" te permitirá ir a otra sala la cual no esté bloqueada, esté adyacente a la actual y exista.

  "Coger objeto" te permitirá agarrar un objeto que exista y esté presente en la sala actual.

  "Soltar objeto" permite colocar un objeto que exista y esté en tu inventario en la sala actual.

  "Usar objeto" permite usar un objeto que exista y esté en tu inventario para desbloquear salidas que requieran de dicho objeto para desbloquearse, estén adyacentes a la actual y existan.

  Y "Resolver puzle / introducir código" permite al usuario resolver un puzle o introducir un código que exista y se encuentre en la sala actual, para así desbloquear las salidas que lo tengan como condición en cualquier parte.

  Si cualquiera de las condiciones especificadas anteriormente no se cumplen, el juego mostrará mensajes de error haciendo saber al jugador lo que está incumpliendo.

- Al pulsar "Guardar partida", los datos de la partida actual son almacenados en el directorio "data" para que el usuario pueda cargar su partida en una próxima sesión.
- Por último, si pulsas "Volver", el usuario volverá a la pantalla de inicio donde podrá crear una Nueva partida, Cargar partida o Salir del juego.

2.6. Guía del operador

(Este programa no requiere de operador)

3. Documentación de sistema

Documentación referida a los aspectos del análisis, diseño, implementación y prueba del *software*, así como la implantación del mismo. En general, debe hacer referencia al sistema, ya que está orientada a los programadores que van a realizar el mantenimiento. Debe estar muy bien estructurada, desde lo más general a los más interno.

3.1. Especificación del sistema

Esta sección debe describir el análisis y la especificación de requisitos. Cómo se descompone el problema en distintos subproblemas y los módulos asociados a cada uno de ellos, acompañados de su especificación. También debe incluir el plan de desarrollo del *software*.

3.2. Módulos

- **Salas**: Este módulo se encarga del administrado tanto de las conexiones como de las salas que componen el escape room. Posee funciones para la carga de conexiones y salas; liberarlas de la memoria, obtener una sala por su ID, y obtener la sala inicial de la partida actual. Este módulo muestra en pantalla errores si no se pudieron cargar los datos de las salas y/o conexiones, o si poseen un formato incorrecto en alguno de sus campos.

- **UI (Interfaz de Usuario)**: Este módulo se centra en pedir información al usuario y mostrarla por pantalla. Posee funciones para la creación de menús, vaciar el buffer de entrada, esperar a que el usuario presione una tecla para continuar, elegir SÍ o NO, mostrar el logo del juego y otros elementos por pantalla; salir del juego, mostrar los menús de registro o inicio de sesión, y el menú principal (el módulo '**partida** usa sus funciones para crear el menú de juego); mostrar los datos de las estructuras cargadas por memoria dinámica con la opción de poner filtros (por ejemplo, para solo ver los objetos en la sala actual); mostrar las interfaces de las diferentes opciones del menú de juego y de salir del juego mostrando unos breves créditos con los miembros creadores del trabajo.

3.3. Plan de prueba

Debe describir todo el plan de pueba que se ha llevado a cabo. Se distinguen las pruebas realizadas a cada módulo y las pruebas de integración de

los distintos módulos probándolo como un todo. Por último, también se debe incluir la descripción del plan de pruebas de aceptación.

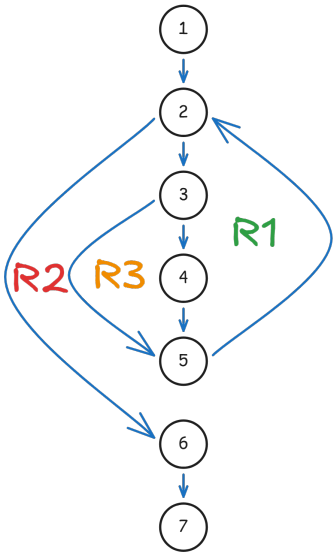
Prueba de los módulos

- **Pruebas de ruta básica de "salas_get_sala_from_id" (José Miguel Pérez Tejero):**

Código:

```
Sala* salas_get_sala_from_id(char* id_to_search, Salas* arr_salas){

    int i;
    Sala* Sala_return = NULL; (1)

    for(i=0; i<(arr_salas->number_of_salas); i++){ (2)

        if(strcmp(arr_salas->salas[i].sala_id, id_to_search) == 0){ (3)

            Sala_return = &(arr_salas->salas[i]); (4)

        } (5)

    } (6)

    return(Sala_return); (7)

}
```

Diagrama de control de flujo:



Complejidad ciclomática:

V(G) = número de regiones (R1, R2 y R3) = 3

Prueba de integración

Incluir toda la batería de pruebas realizadas. Pruebas de caja blanca y pruebas de caja negra.

Plan de pruebas de aceptación

Estas pruebas deben diseñarse con el usuario del sistema. Deben describir las pruebas que son necesarias pasar para que el sistema sea aceptado por el usuario final.

Si en algún punto del documento se quiere hacer referencia a algún documento es necesario incluir la cita donde corresponda, podemos tomar como ejemplo **?**.

4. Documentación del código fuente

En este punto se puede incluir toda la documentación interna que puede ser generada con Doxygen.

La documentación interna del programa, como se ha comentado anteriormente, comprende los comentarios del programa. Durante la fase de implementación es necesario comentar adecuadamente cada una de las partes del programa. Estos comentarios se incluyen en el código fuente con el objetivo de aclarar qué es lo que hace el fragmento de código, para explicar sus elementos.

Esta documentación ayuda a hacer más comprensible el código fuente a los programadores que deban trabajar en él, facilitando las tareas de mantenimiento. No sólo proporciona una mayor legibilidad en la actualización y modificación del código, sino también en la depuración del mismo.